Informe de Pràctica:
Sistema de Diàleg

Tractament de la Veu i el Diàleg, GIA

Octubre de 2026

Irene Miralles i Berta Vidal

Índex

1. Introducció 3

2. Sistema basat en regles 3

2.1. Sistema inicial i diagnòstic 3

2.2. Objectius i decisions de disseny 4

2.3. Implementació 5

2.3.1. Comprensió del llenguatge 5

2.3.2. Gestió del diàleg 5

2.3.3. Generació de la resposta 6

2.4. Proves i resultats 6

2.5. Conclusions 7

3. Sistema basat en LLM i eines 8

3.1. Sistema inicial i diagnòstic 8

3.2. Objectius i decisions de disseny 9

3.3. Implementació 9

3.3.1. L'eina de cerca (filter_real_estate) 9

3.3.2. Guardrail d'entrada (InputGuardrail) 10

3.3.3. Guardrail d'execució (ExecutionGuardrail) 10

3.3.4. Guardrail de sortida (OutputGuardrail) 10

3.4. Proves i resultats 11

3.5. Conclusions 11

4. Annexos 13

4.1. Annex A: Diagrama de flux del sistema basat en regles 13

1. Introducció

En aquesta pràctica hem dissenyat i implementat un sistema de diàleg recomanador d'habitatges basat en una base de dades en format JSON. L'objectiu principal d'aquest treball ha estat explorar i comparar dos paradigmes fonamentals en el desenvolupament de sistemes de conversa: en primer lloc, un sistema determinista basat en regles i, en segon lloc, un sistema basat en un model de llenguatge de gran escala (LLM) equipat amb eines (*tools*).

Aquest document recull la metodologia de treball, les decisions de disseny adoptades, el procés d'implementació i les conclusions extretes al llarg del desenvolupament dels dos sistemes.

Per desenvolupar tots dos sistemes hem seguit un mètode iteratiu. Hem partit d'una versió inicial, l'hem provada amb diàlegs que inclouen casos habituals i casos límit, hem identificat els errors i hem provat solucions. A partir d'aquí, hem repetit el procés de prototipat, implementació i proves diverses vegades, comprovant que les actualitzacions funcionaven i que no generaven errors nous. Per això, per a cada sistema expliquem primer el punt de partida i els problemes detectats; després, els objectius i les decisions de disseny i la implementació de les solucions, i, finalment, les proves, els resultats i les conclusions.

2. Sistema basat en regles

Un sistema basat en regles és una arquitectura determinista en què el flux de la conversa, la interpretació del llenguatge i la selecció de respostes estan regits per un conjunt explícit de regles lògiques, patrons sintàctics i diccionaris predefinits. A continuació, expliquem el desenvolupament i la millora progressiva del nostre sistema basat en regles.

2.1. Sistema inicial i diagnòstic

El sistema basat en regles ha partit de l'assistent immobiliari desenvolupat en els exercicis inicials de la pràctica. Aquest sistema base funcionava com un formulari seqüencial rígid que recorria un conjunt de preguntes definides en un fitxer JSON, d'una en una i en un ordre fix, fins a conèixer les preferències per a un seguit d'atributs establerts (tipus, zona, pressupost, habitacions, banys, etc.). Només acceptava respostes de múltiple opció que coincidissin exactament amb el text esperat o, en el cas dels números, n'extreia exclusivament el primer *token* numèric detectat. El filtratge dels habitatges s'executava únicament al final del procés, i mostrava els resultats sense cap ordenació o emetia un missatge genèric d'error (*“Sorry, no suitable houses...”*).

Per diagnosticar l'estat d'aquest sistema inicial, hem executat un conjunt divers de diàlegs de prova amb entrades realistes i casos límit. A partir d'aquí, hem identificat un seguit de limitacions clau, que hem agrupat en tres problemàtiques principals:

1. **Comprensió:** Hem comprovat que el sistema no oferia cap tolerància a les faltes d'ortografia, no reconeixia àlies d'ubicacions ni barris (p. ex., “L'Hospitalet” vs. “Hospitalet”, o “Gràcia”), ni entenia els números escrits en lletres (“dues”, “mil”) ni els separadors de milers o decimals. Tampoc processava adequadament la magnitud dels preus, ignorava els operadors lògics (“com a màxim”, “almenys”), no interpretava negacions (“sense ascensor”) ni processava correctament múltiples dades o operacions complexes dins d'una mateixa frase.
2. **Robustesa:** Hem detectat que el programa s'aturava abruptament (llançant excepcions no gestionades com *ValueError*) davant d'entrades inesperades amb unitats (com “80m” o “2h”), acceptava valors completament fora de rang i entrava en bucles repetitius sense explicar l'error a l'usuari.

3. **Rigidesa del diàleg:** També hem comprovat que l'ordre fix de preguntes obligava a demanar atributs innecessaris, que no existia cap mecanisme de confirmació implícita d'allò que s'havia entès i que l'usuari no podia corregir ni modificar una preferència prèvia. A més, si no es trobava cap resultat no s'oferia cap alternativa, i els resultats finals es mostraven sense cap ordenació ni paginació.

2.2. Objectius i decisions de disseny

Per donar resposta a les limitacions detectades, hem establert sis objectius de disseny per al nou sistema:

1. **Diàleg d'iniciativa mixta:** L'assistent ha de portar el fil de la conversa, però l'usuari ha de tenir la llibertat d'aportar, corregir o indicar com a indiferent (ANY) qualsevol criteri en qualsevol torn, així com de fer preguntes puntuals sobre un pis concret.
2. **Pregunta inicial oberta i preguntes adaptatives:** La conversa s'ha d'iniciar amb una pregunta oberta que permeti a l'usuari expressar tot el que desitgi d'una sola vegada. Després, el sistema ha de formular preguntes sobre un sol atribut desconegut a la vegada.
3. **Actualització dinàmica de candidats:** A cada torn s'han de recalcular els pisos candidats en funció de la informació acumulada, i la pregunta següent s'ha d'adaptar als candidats restants.
4. **Comprensió avançada i tolerant:** El sistema ha d'interpretar faltes d'ortografia, variacions lingüístiques (català, castellà i anglès), números en lletres, multiplicadors, operadors lògics, intervals i múltiples dades en una sola frase.
5. **Generació clara, natural i adaptada:** La resposta s'ha de generar en català natural, comunicant només la informació necessària, utilitzant connectors lògics adequats i gestionant de manera específica la recuperació d'errors.
6. **Robustesa:** L'execució del programa no s'ha d'aturar mai, independentment de l'entrada de l'usuari.

Per assolir aquests objectius hem pres les decisions de disseny següents:

1. **Fitxa de preferències:** Les preferències es representen com una fitxa d'atributs, en què cada atribut pot estar en tres estats: desconegut, amb un valor o indiferent. Distingir entre desconegut i indiferent ens permet preguntar només el que és desconegut i que l'usuari pugui omplir o corregir qualsevol atribut des de qualsevol frase.
2. **Ordre de les preguntes:** L'ordre de prioritat de les preguntes es defineix al fitxer JSON i comença pel tipus d'operació, perquè determina si cal preguntar el pressupost de compra o els ingressos (per als lloguers). Si el tipus és indiferent, es pregunten tots dos.
3. **Criteris de filtratge:** Hem mantingut en gran mesura els criteris de l'enunciat i dels exercicis anteriors:
   – Habitacions, banys, superfície i planta filtren per un mínim.
   – El preu filtra per un màxim.
   – El tipus d'operació filtra per igualtat, i la zona, per pertinença.
   – En el cas del lloguer, el preu no pot superar el 35 % dels ingressos mensuals.
   – Respondre “no” a terrassa, ascensor o ús comercial vol dir que és indiferent; per tant, no s'exclouen els pisos que en tenen.
4. **Zones:** La zona es tracta com un conjunt. L'usuari pot dir “Barcelona o Esplugues” o “qualsevol menys Santa Coloma”, i afegir-ne o treure'n més endavant. Si l'usuari demana un barri de Barcelona, el sistema ho entén com a “Barcelona”.

5. **Imports:** Quan l'usuari dona un import sense haver especificat el tipus d'operació, el sistema no l'assigna directament: en fa una hipòtesi segons la magnitud i demana confirmar-la.
6. **Confirmació:** Cada vegada que l'usuari dona, canvia o descarta un criteri, el sistema confirma implícitament el que ha entès ("D'acord, busco pisos de compra a Esplugues"), perquè l'usuari pugui detectar un error i corregir-lo de seguida. Quan ja no queden preguntes, fa un resum de tots els criteris abans de mostrar els resultats.

2.3. Implementació

Aquest apartat descriu com hem resolt les limitacions descrites a l'apartat 2.1 i els problemes que han aparegut en les diferents proves. Les solucions s'organitzen en tres blocs: comprensió del llenguatge, gestió del diàleg i generació de la resposta. El diagrama de flux del sistema final es troba a l'annex A.

2.3.1. Comprensió del llenguatge

Cada frase de l'usuari passa per un conjunt de regles que n'extreuen la informació. Primer es normalitza (minúscules, sense accents i amb els apòstrofs unificats) i les paraules desconegudes es corregeixen comparant-les amb el lèxic del sistema. Després es converteixen els números escrits en lletres i els formats habituals, i cada número s'assigna a l'atribut de la paraula clau que l'acompanya, tenint en compte els operadors ("com a màxim", "almenys") i els intervals; si no en té cap, s'assigna a l'atribut que s'acaba de preguntar. Amb llistes de paraules clau es detecten també el tipus d'operació, les zones incloses i excloses, les característiques amb negació ("sense ascensor") i les intencions de l'usuari. Els valors fora dels rangs definits al fitxer JSON es tracten com a errors, i tot el lèxic inclou el català, el castellà i l'anglès.

2.3.2. Gestió del diàleg

A cada torn, el model de decisió fa una sèrie de comprovacions en un ordre fix, que defineixen els camins possibles del diàleg:

1. Primer atén les peticions de sortir, d'ajuda o de nova cerca, i els errors de comprensió. Els errors s'expliquen de manera concreta i, a partir del segon intent fallit, amb exemples de resposta.
2. Si hi ha un import pendent de confirmar, demana el tipus d'operació.
3. Si amb els criteris actuals no hi ha cap pis, ho avisa al moment i proposa fins a dos canvis mínims que donarien resultats, com ara rebaixar el nombre d'habitacions o pujar el pressupost.
4. Si tots els candidats tenen el mateix valor per al següent atribut, no el pregunta i ho comunica.
5. Si cap candidat té una característica, no la pregunta: ho comunica i, si escau, proposa canviar algun criteri.
6. Quan queden tres pisos o menys, ofereix mostrar-los i, alhora, fa la pregunta següent, per si l'usuari vol afinar més la cerca.
7. En qualsevol altre cas, pregunta el següent atribut desconegut segons l'ordre de prioritat definit al JSON.

Els resultats es mostren ordenats per preu, de tres en tres. Després, l'usuari pot demanar-ne més, veure els detalls d'un pis, preguntar-ne una dada, canviar qualsevol criteri o començar una nova cerca.

En provar aquesta versió amb diàlegs nous, hem detectat que el sistema no tenia memòria de la conversa. Per exemple:

- Una pregunta com “la casa 19 té ascensor?” s'interpretava com un criteri nou i canviava la cerca.
- Una pregunta com “i té terrassa?” no es podia relacionar amb el pis consultat.
- Un requisit que no és a les dades, com “amb piscina”, s'ignorava sense avisar.

Per resoldre-ho, hem ampliat el model de context. La classe *HouseBot*, que gestiona el diàleg, ara guarda a cada torn l'últim pis consultat i la darrera consulta, què s'espera que respongui l'usuari, els requisits que no es poden comprovar i els avisos que ja s'han donat. L'ordre “nova cerca” esborra tota aquesta informació amb la funció *reset()*.

Amb aquest context, el sistema pot distingir una consulta d'un canvi de preferència. Una pregunta com “la casa 19 té ascensor?” es respon sense modificar la cerca i acaba amb una proposta perquè l'usuari decideixi si vol afegir aquell atribut als requisits. També pot resoldre les referències: diferencia l'identificador d'un pis de la seva posició a la llista mostrada, relaciona “aquest pis” o “i té terrassa?” amb l'últim pis consultat i, si la referència és ambigua, demana aclariment.

Els requisits que no es poden comprovar es detecten a partir d'una llista tancada del lèxic amb característiques que no són a les dades, com la piscina, l'aparcament o el jardí. La funció *interpret*, que processa l'entrada, els identifica abans d'extreure la resta d'informació. Aquests requisits no s'apliquen com a filtre, perquè no es poden comprovar. El sistema avisa l'usuari una sola vegada, ho torna a recordar quan mostra els resultats i, si es pregunta per un pis concret (“la casa 19 té piscina?”), respon que no en té informació, en lloc de dir que el pis no en té.

Finalment, quan el sistema no pot atendre una petició, ho explica i proposa com continuar. Ho fa de manera diferent segons el motiu: si no ha entès l'entrada, si la dada no és a la base de dades, si es demana una acció que no pot fer (com programar una visita) o si no sap a quin pis es refereix l'usuari.

2.3.3. Generació de la resposta

Per millorar la generació de la resposta, l'hem separada en dues fases. La microplanificació, a les funcions *plan_events* i *plan_action*, decideix què es diu: selecciona només la informació necessària, sense escriure encara cap frase. Per exemple, una consulta es respon només amb el que s'ha preguntat, i en una llista de resultats el que comparteixen tots els pisos es diu una sola vegada. La realització, a la funció *realize*, decideix com es diu: converteix aquesta informació en text i hi afegeix els connectors adequats. Finalment, la funció *orthography* corregeix els errors que poden aparèixer en combinar plantilles, com les contraccions i l'apostrofació.

2.4. Proves i resultats

Per validar el sistema hem desenvolupat un conjunt de 72 proves automàtiques. Cada prova reprodueix una conversa amb el sistema i verifica que el resultat és l'esperat; si durant la conversa es produeix algun error d'execució, la prova es considera fallida. Les proves s'organitzen en tres blocs.

En primer lloc, un bloc avalua la comprensió del llenguatge. S'hi valida:

- La tolerància a majúscules, faltes d'ortografia i sinònims en català, castellà i anglès.
- El reconeixement de les zones encara que s'escriguin de maneres diferents (“Hospitalet” o “L'Hospitalet de Llobregat”), es demani un barri de Barcelona o una zona on no hi ha pisos, o se n'indiquin diverses alhora, incloent-hi exclusions (“qualsevol menys Santa Coloma”).
- La interpretació de números escrits en lletres, separadors de milers, multiplicadors (“250 mil”, “300k”) i ingressos anuals.

- L'extracció de diverses dades en una sola frase, amb operadors, intervals i negacions.
- El tractament de valors fora de rang, decimals i unitats inesperades sense que el programa s'aturi.

En segon lloc, un altre bloc avalua la gestió del diàleg. En concret, comprovem:

- La formulació de preguntes només sobre el que el sistema encara no sap, amb una confirmació breu del que ha entès i la possibilitat de corregir qualsevol criteri en qualsevol moment.
- La confirmació del tipus d'operació quan l'usuari indica un import sense dir si és de compra o de lloguer.
- L'adaptació als pisos disponibles: l'avís immediat quan no en queda cap, amb una proposta de canvi; l'omissió de les preguntes en què tots coincideixen, i l'oferta de mostrar-los quan en queden pocs.
- La presentació dels resultats ordenats per preu i de tres en tres, amb la possibilitat de veure'n més, demanar els detalls d'un pis, canviar un criteri o començar una nova cerca.
- L'ús del context de la conversa: consultes sobre un pis que no modifiquen la cerca, referències com “aquest pis” o “la segona”, avisos sobre requisits que no es poden comprovar i explicacions quan una petició no es pot atendre.

Per últim, el tercer bloc avalua la generació de la resposta. S'hi comprova que només es diu la informació necessària, que els detalls es redacten amb connectors i que les plantilles es combinen sense errors ortogràfics.

A més, un revisor automàtic analitza totes les respostes de les proves a la recerca d'errors de combinació de plantilles, i una prova de robustesa executa diàlegs aleatoris amb entrades vàlides, invàlides i sense sentit. El sistema final supera totes les proves, el revisor no detecta cap error en les respostes i la prova de robustesa no genera cap excepció.

2.5. Conclusions

En aquesta primera tasca hem transformat un formulari seqüencial i rígid en un diàleg d'iniciativa mixta. El sistema final permet a l'usuari expressar-se lliurement, tolera faltes i variacions lingüístiques, només pregunta el que és útil segons els pisos disponibles i genera respostes concises i naturals. La separació entre comprensió, gestió del diàleg i generació, i dins d'aquesta última entre microplanificació i realització, ens ha permès millorar cada part de manera independent.

Tot i això, el sistema té les limitacions pròpies d'una aproximació basada en regles. Només entén el que preveu el lèxic, de manera que expressions no previstes com “una cosa assequible” s'ignoren. A més, les consultes es detecten per patrons: una pregunta formulada d'una manera inesperada es pot interpretar malament. Cada comportament nou requereix escriure regles noves i provar-les, i per això el cost de manteniment creix a mesura que el sistema es fa més complet.

A canvi, és un sistema completament predictible: davant de la mateixa entrada sempre respon igual, no pot inventar dades i cada comportament es pot provar de manera automàtica.

3. Sistema basat en LLM i eines

La segona tasca de la pràctica ha consistit a desenvolupar el mateix xatbot recomanador d'habitatges, però fent servir un LLM equipat amb eines. Un sistema basat en un LLM amb eines és una arquitectura en què el model de llenguatge actua com a base per comprendre l'usuari i dialogar-hi, però es recolza en funcions de codi deterministes per dur a terme tasques precises que el model no pot fer de manera fiable per si mateix, com ara aplicar filtres exactes sobre una base de dades. A continuació, expliquem el desenvolupament i la millora progressiva del nostre sistema basat en LLM.

3.1. Sistema inicial i diagnòstic

El punt de partida d'aquesta segona tasca ha estat un xatbot bàsic en anglès format pel model *Qwen/Qwen2.5-Coder-3B-Instruct* i l'eina externa *filter_real_estate*, encarregada de fer consultes sobre el conjunt de dades en format JSON (*house_data_LLM.json*).

En aquest disseny inicial, el flux era directe i sense intermediaris: l'usuari introduïa la seva petició, el model l'analitzava i, si necessitava cercar habitatges, generava una petició en format JSON amb els arguments de la funció. La funció *query_model_with_tools* extreia aquest JSON mitjançant expressions regulars, executava l'eina de filtratge sobre la base de dades i afegia els resultats en brut (un altre JSON) a l'historial del model. Finalment, el model llegia els resultats i generava una resposta per a l'usuari.

Tot i complir l'objectiu bàsic, en les proves de diagnòstic hem detectat els problemes següents, que calia resoldre per aconseguir un sistema segur i funcional:

1. **Vulnerabilitat a injeccions de prompt:** Com que el text de l'usuari es fusionava directament amb el prompt de sistema sense cap fase de control, el sistema era molt vulnerable. Hem comprovat que un usuari podia enganyar fàcilment el model per fer-li ignorar les instruccions, revelar el prompt intern o adoptar comportaments fora de l'àmbit immobiliari.
2. **Al·lucinacions i mala extracció de paràmetres:** En tractar-se d'un model de mida moderada (3.000 milions de paràmetres), hem observat una tendència recurrent a cometre errors en traduir el llenguatge natural als arguments exactes de la funció de filtratge. Això es traduïa en:
   - **Al·lucinació d'atributs:** el model s'inventava característiques o criteris no esmentats per l'usuari ni existents a la base de dades.
   - **Errors de format i rang:** es generaven paràmetres amb tipus de dades incoherents o amb valors numèrics fora dels rangs permesos per l'eina.
3. **Absència de gestió de l'estat del diàleg i de les cerques buides:** El sistema inicial no disposava d'una memòria estructurada per mantenir els criteris acumulats al llarg de la conversa, de manera que el model perdia el context en diàlegs de diversos torns o repetia preguntes. A més, quan la cerca retornava 0 resultats, el model no sabia com gestionar la situació ni proposava estratègies per relaxar els filtres.
4. **Manca de controls:** Tot el control del flux, la seguretat i la validació de dades depenia exclusivament de la capacitat generativa del model. Sense capes deterministes de verificació (*guardrails*) a l'entrada ni a la sortida, qualsevol desviació de l'LLM derivava directament en una fallada en l'execució del codi o en una resposta inadequada.

3.2. Objectius i decisions de disseny

Per abordar les limitacions del sistema inicial, hem redefinit l'arquitectura amb l'objectiu de transformar el prototip en un assistent immobiliari robust, segur i coherent. Les decisions de disseny clau que hem pres són les següents:

1. **Rol i llengua:** Hem establert que l'assistent actuï de manera constant com un consultor immobiliari professional, educat i directe, que s'expressa per defecte en català.
2. **Separació de responsabilitats:** Hem delegat les tasques deterministes (com la cerca de valors o la validació de rangs) a mòduls de codi en Python, i hem reservat l'LLM per a la comprensió del context i la redacció de respostes naturals.
3. **Arquitectura de protecció:** Per protegir el sistema contra el mal ús i les fallades, hem implementat tres capes de guardrails:
   – **Capa d'entrada (Input Guardrail):** intercepta atacs o preguntes alienes abans de processar-les.
   – **Capa d'execució (Execution Guardrail):** valida els filtres abans de consultar la base de dades.
   – **Capa de sortida (Output Guardrail):** revisa la resposta redactada abans de mostrar-la a l'usuari.
4. **Gestió de l'estat del diàleg:** Hem introduït una memòria estructurada (la classe *ChatState*) que emmagatzema les preferències de l'usuari (pressupost, ciutat, habitacions, etc.) al llarg de la conversa, de manera que l'assistent no oblida informació ni repeteix preguntes.
5. **Cerca flexible i gestió de resultats buits:** Hem dissenyat un mecanisme que reactiva la cerca quan l'usuari canvia de criteri i que, quan no hi ha coincidències exactes (0 resultats), proposa automàticament alternatives relaxant les restriccions.

3.3. Implementació

A continuació detallem com hem implementat la capa de codi que envolta l'LLM, formada per l'eina de filtratge i els tres nivells de guardrails.

3.3.1. L'eina de cerca (*filter_real_estate*)

Hem definit una única eina a disposició del model, que actua com a interfície entre el llenguatge natural i la base de dades. Per implementar-la, hem pres les mesures següents:

- **Connexió amb el sistema de regles:** L'eina no filtra per si mateixa, sinó que recull els arguments de l'LLM i crida la funció determinista desenvolupada en la primera tasca (*find_suitable_houses*). Això garanteix que els filtres complexos, com ara que el preu del lloguer no superi el 35 % dels ingressos de l'usuari, s'apliquin de manera exacta.
- **Traducció i format per a l'LLM:** Per reduir la càrrega del model i evitar errors de format en la resposta final, l'eina tradueix el JSON intern abans de retornar-lo al model: les claus es passen al català i els valors inclouen les unitats corresponents.
- **Límit de resultats i propostes de relaxació:** Hem limitat el retorn a un màxim de cinc habitatges per no desbordar la finestra de context de l'LLM; l'assistent n'informa l'usuari i li permet continuar filtrant. Si la cerca retorna 0 resultats, l'eina calcula quants pisos hi hauria si s'eliminés un dels criteris i afegeix aquests suggeriments al JSON de resposta perquè el model els pugui oferir a l'usuari.

3.3.2. Guardrail d'entrada (*InputGuardrail*)

Aquest primer filtre analitza el text de l'usuari mitjançant expressions regulars abans que arribi al model:

- **Prevenció d'injeccions de prompt i filtracions:** Hem definit patrons per bloquejar missatges que intenten alterar el rol del sistema (p. ex., “ignora les instruccions”, “actua com a...”) o que demanen revelar el prompt de sistema o la configuració interna de les eines. També bloquegem intents d'injecció de codi JSON o crides directes a l'eina (*"name": "filter_real_estate"*).
- **Control de l'àmbit de la conversa:** Hem incorporat regles per detectar i desviar consultes fora del domini, que es responen amb un missatge informatiu sense consumir recursos del model.
- **Detecció d'atributs no disponibles:** Si l'usuari demana característiques que no són a la base de dades (piscina, jardí, vistes, pàrquing, etc.), el sistema elimina aquesta part de la frase i informa l'usuari de les dades que hi ha realment disponibles.

3.3.3. Guardrail d'execució (*ExecutionGuardrail*)

Aquesta capa valida el JSON d'arguments que l'LLM genera per cridar l'eina. El seu objectiu és evitar que el model s'inventi paràmetres o n'alteri els valors:

- **Extracció directa:** Hem implementat funcions de processament de text (*extract_numbers*, *parse_slots*, *locations_in_text*) que llegeixen el missatge de l'usuari en paral·lel al model. Si l'LLM intenta cridar l'eina amb un nombre o un municipi que l'usuari no ha dit, la crida es rebutja automàticament.
- **Validació de dades i domini:** Verifiquem que els arguments estiguin dins dels rangs vàlids. A més, resolem les deduccions que l'LLM no sap fer, com ara multiplicar preus per mil (quan l'usuari diu “250k”, o “250” per a una compra) o interpretar un sou de “30.000” com a anual.
- **Control de l'estat:** Emmagatzemem els criteris vàlids a l'estat de la sessió. Així, les cerques posteriors mantenen els criteris anteriors, tret que l'usuari els canviï explícitament. Si a la crida li falten dades essencials (com saber si és compra o lloguer), l'execució es bloqueja i es demana l'aclariment a l'usuari.

3.3.4. Guardrail de sortida (*OutputGuardrail*)

Un cop l'LLM ha llegit les dades de l'eina i ha redactat una resposta, aquesta passa per un últim control abans de mostrar-se a l'usuari:

- **Verificació de la llengua i de les promeses:** Una heurística comprova que el text generat sigui en català. A més, rebutgem la sortida si s'hi detecten promeses que no es poden complir (descomptes, aprovació de reserves o gestió de visites).
- **Detecció d'al·lucinacions numèriques i descriptives:** El guardrail comprova que la resposta no inclogui atributs inventats (com “pis molt lluminós” o “barri tranquil”) i que tots els nombres del text corresponguin a un valor introduït per l'usuari o retornat per l'eina.
- **Mecanisme de regeneració:** Si el guardrail detecta un error, indica al model quin és (p. ex., “has esmentat el número 4, però no és a les dades”) i li demana que reescrigui la resposta. Si després d'un parell d'intents continua fallant, el sistema prescindeix del model i retorna una resposta segura generada amb una plantilla determinista.

3.4. Proves i resultats

Per validar l'arquitectura i comprovar que els guardrails funcionaven correctament, hem desenvolupat un conjunt de proves automàtiques. Aquestes proves simulen converses complexes i els casos límit detectats en el diagnòstic inicial. A més, al final de cada torn mostrem per pantalla l'acció interna de cada capa (*show_audit()*). Aquest registre ens ha permès veure quins guardrails s'activaven en cada moment i per què es corregien o descartaven els paràmetres, cosa que ha facilitat la diagnosi i la correcció del sistema.

Igual que en la primera tasca, les proves s'organitzen en tres blocs.

En primer lloc, un bloc avalua la comprensió, l'extracció de dades i l'aplicació de regles. S'hi valida:

- La gestió de diàlegs llargs i l'extracció de múltiples filtres en un únic missatge.
- L'aplicació correcta de regles complexes, com el límit del lloguer al 35 % dels ingressos mensuals.
- La resposta davant de peticions de característiques que no són a la base de dades (com piscines, jardins o la ubicació per barris), comprovant que el guardrail les neutralitza sense permetre al·lucinacions.
- El multilingüisme d'entrada, verificant que el model comprèn paràmetres expressats en anglès i català, però respon sempre en català.

En segon lloc, un altre bloc avalua la seguretat del sistema davant d'atacs. En concret, comprovem:

- La robustesa davant d'injeccions de prompt que intenten que el model canviï de rol, accepti condicions absurdes o aprovi transaccions irreals (p. ex., vendre una casa per 1 euro).
- La protecció contra filtracions (*prompt leaks*), assegurant que el model no revela les instruccions de sistema, els guardrails interns ni el funcionament de l'eina.
- La desviació de temes fora de l'àmbit (programació, consells de salut, hipoteques o recomanacions bancàries).
- El bloqueig d'intents d'injecció de codi i de crides artificials a l'eina des del xat (com forçar la lectura de rutes de fitxers), així com de peticions explícites de resposta en format JSON brut.

Per últim, el tercer bloc avalua funcionalitats més específiques i les modificacions introduïdes per resoldre els problemes que hem anat detectant.

Hem executat aquesta bateria de proves en cada iteració fins que el sistema final les ha superades totes amb el comportament esperat. La combinació dels tres nivells de guardrails fa que l'assistent apliqui correctament els filtres de cerca i bloquegi els atacs abans que arribin al model. Fins i tot quan l'LLM intenta al·lucinar arguments en la crida a l'eina, el mecanisme de regeneració fa que l'usuari rebi una resposta útil i fidel a les dades.

3.5. Conclusions

En aquesta segona tasca hem pogut avaluar el salt qualitatiu que suposa integrar un LLM respecte al sistema basat en regles. La incorporació de l'LLM ha dotat l'assistent de capacitat de comprensió semàntica, cosa que aporta una gran flexibilitat conversacional: l'usuari es pot expressar de manera desordenada, amb llenguatge col·loquial i amb formulacions que no preveu cap lèxic. El sistema basat en regles ja admetia diversos criteris en una sola frase i tres idiomes, però només dins del que recollien les regles; l'LLM elimina aquesta dependència del lèxic, que era la principal limitació identificada a la primera tasca.

Alhora, hem comprovat les limitacions inherents a aquesta arquitectura. A causa de la seva naturalesa probabilística, el prompt de sistema per si sol no evita el risc d'al·lucinacions ni les vulnerabilitats de seguretat. Hem pogut constatar la importància d'implementar controls i de fer servir eines que superin les limitacions d'un LLM de mida moderada com el que hem emprat. Així doncs, l'èxit de l'assistent final rau en l'arquitectura que envolta el model: els guardrails d'entrada, d'execució i de sortida, i una eina capaç de fer consultes deterministes i filtres exactes sobre la base de dades.

D'altra banda, aquesta aproximació també té limitacions pel que fa a la càrrega computacional i la finestra de context. Les crides contínues al model i les reavaluacions de la capa de sortida incrementen la latència de resposta, i la finestra de context obliga a restringir el nombre de resultats que retorna l'eina per no saturar la memòria ni degradar la precisió de l'LLM.

Finalment, en comparar les dues tasques, concloem que la millor aproximació per a l'assistent immobiliari seria un sistema combinat que aprofités els avantatges de totes dues arquitectures: utilitzar l'LLM exclusivament com a interfície flexible de comprensió i generació del llenguatge, i delegar en funcions deterministes, predictibles i que es poden provar automàticament, la cerca, la memòria d'estat, l'aplicació de regles i també la gestió del diàleg, tal com fa el sistema de la primera tasca.

4. Annexos

4.1. Annex A: Diagrama de flux del sistema basat en regles

Aquest és el diagrama de flux que representa el comportament del sistema basat en regles.

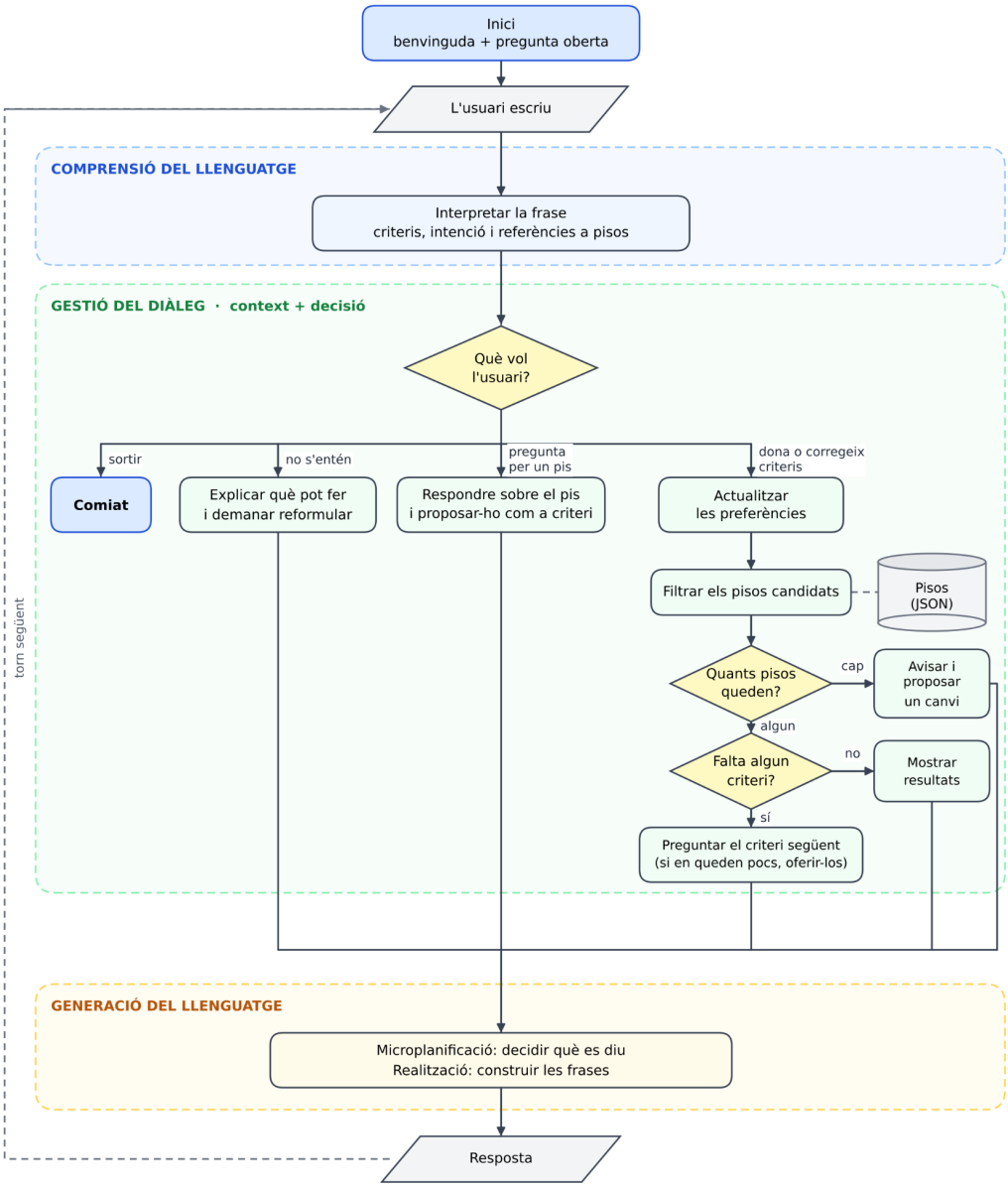


Figura 1. Diagrama de flux del sistema basat en regles